

MICROSOFT · HANDS-ON LAB WORKSHOP

Agent Hosting on Azure

Three production patterns for running stateful AI agents on Azure



Foundry Host Agent



AKS + agent-sandbox



ACA Sandbox

Deploy the same agent three ways · compare isolation, scale-to-zero, identity & cost · ~135 minutes



How Today Works

Six short modules — one shared foundation, three solution labs, one wrap-up

0

Introduction

10 min · Framing & architecture — you are here

1

Core Infrastructure

30 min · Deploy once — shared by all three

2

Solution A — Foundry Host Agent

30 min · Managed on-ramp, deployed with azd

3

Solution B — AKS + agent-sandbox

40 min · Kata Container micro-VM isolation

4

Solution C — ACA Sandbox

20 min · Serverless, micro-VM isolation

5

Wrap-up & Decision Guide

5 min · Compare, cost tips & Q&A



Before We Start

Prerequisites & setup check — fix any gaps now while we cover concepts



Accounts & Access

- Azure subscription — Contributor access
- Module 1 role assignments need Owner or User Access Administrator
- Role: Foundry User (for the hosted agent)
- Role: Container Apps SandboxGroup Data Owner



Tools to Install

- Azure CLI — az login completed
- Azure Developer CLI (azd) + Foundry extension
- kubectl and Docker (for Solution B)
- ACA preview extension (containerapp --allow-preview)



Most common blocker: creating role assignments needs Owner or User Access Administrator — plain Contributor can't. Confirm your access now.



Why Host Agents on Azure?

Production agents are stateful services — not stateless API calls

A demo agent is easy. A production agent must remember, isolate, authenticate, govern, and stay cheap when idle.



Isolation

Tenant / user boundaries



Scale-to-zero

No cost while idle



Identity & Auth

Managed identity, no secrets



Governance & Cost

One gateway, predictable spend

One size doesn't fit all — enterprise and consumer priorities differ. This workshop = three patterns for the same agent.



What Is a “Hosted Agent”?

The contract every solution in this workshop honors



Persistent context

Remembers the conversation across many turns



Managed-identity auth

Authenticates to the model with an Azure identity — no static secrets



Scale-to-zero + restore

Hibernates when idle, reloads state on the next request



All LLM calls via APIM

Every model call routes through one governed AI Gateway

Whatever the hosting technology, the agent behaves the same to a caller.



Two Target Scenarios

Enterprise vs. consumer — the personas that drive every design choice



ToB — Enterprise

Business / internal platforms

- Strong tenant & department isolation + audit
- Entra ID — SSO, RBAC, Conditional Access
- Data residency, private networking, compliance
- Reserved or burstable, predictable capacity

Security · Governance · Reliability



ToC — Consumer

Individual end-users / small teams

- Very large volume of short-lived sessions
- Process- or container-level isolation (lighter)
- Social login (External ID / B2C) or API key
- Aggressive scale-to-zero, pay-per-use

Cost · Speed · Simplicity



Three Solutions at a Glance

The whole day in one view — same agent, three hosts



Foundry Host Agent

BEST FOR

ToB managed · fastest on-ramp

ISOLATION

Managed (per-agent)

GA



AKS + agent-sandbox

BEST FOR

ToB high-security

ISOLATION

Micro-VM (Kata)

GA



ACA Sandbox

BEST FOR

ToC / ToB long-running

ISOLATION

Micro-VM

Public Preview

What changes is isolation & control — Blob state and the APIM AI Gateway stay constant across all three.



Deploy Once: Core Infrastructure

Module 1 — the shared foundation all three solutions reuse



Shared platform

Resource Group

Blob Storage

APIM (Basic v2)

Key Vault

Container Registry

Entra App Reg

User-Assigned MI



Foundry stack

Foundry account

Project maf-agent-prj

gpt-5.4-mini

Defender for AI

2 RAI policies

APIM AI gateway

One deploy: `setup.sh` → `main.bicep` → `core.bicep` · Modules B & C discover it via the `deploymentSN` tag — reuse, never recreate.



The State Pattern

Azure Blob is the single source of truth — this is why scale-to-zero is free



New request

Load state from Azure Blob (per-agent JSON)



Active session

Save to Blob on every change (each turn)



Scale-to-zero

Nothing to flush — state already durable



Single source of truth: Azure Blob Storage. Each agent stores `<AGENT_ID>.json` in the `agent-state` container. No Redis / hot cache. Cool tier + versioning = cheap and recoverable.



Security: Identity + AI Gateway

No static keys anywhere — APIM is the one governed front door



Scope: user → agent authentication is out of scope today. We secure the agent → model hops.

Path 1 — all solutions

- Agent generates an Entra ID token
- → APIM AI API (validate-jwt)
- → APIM calls the LLM with its UAMI

Path 2 — Solution A only

- Hosted agent → its Foundry project
- (Foundry-assigned identity / Foundry User)
- → APIM AI gateway → LLM

APIM = validate-jwt, rate limits, token quotas, logging, and it hides the Foundry endpoint.



Solution A — Foundry Host Agent

ToB Managed · fastest on-ramp · uses BOTH Path 1 and Path 2

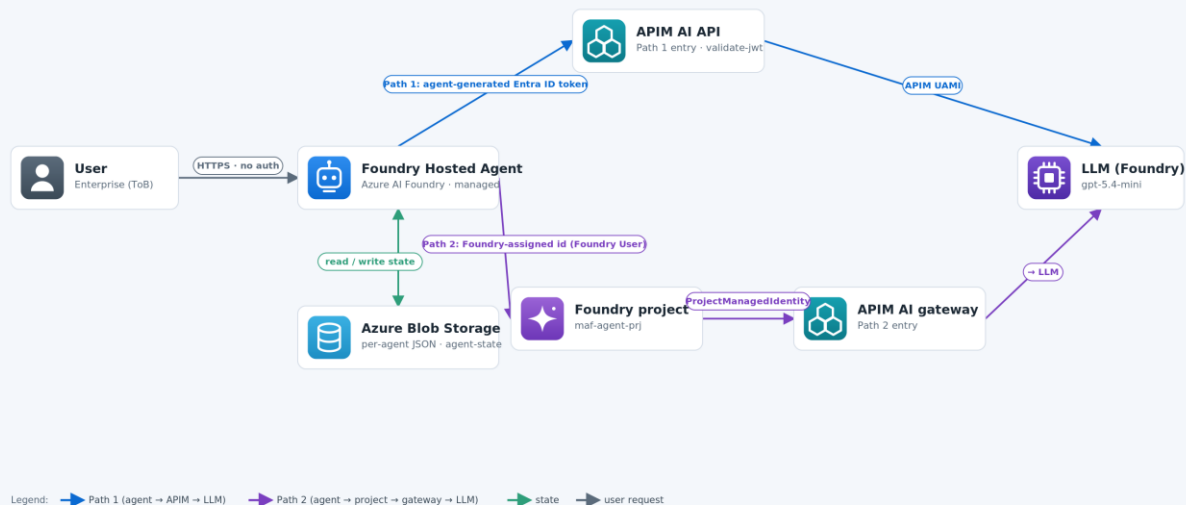
AT A GLANCE

- Fully managed agent runtime (Azure AI Foundry)
- Deploy with azd — no new infra to stand up
- Model routing: direct or gateway (one env var)
- Runs on gpt-5.4-mini via the APIM AI Gateway
- Status: GA · Best for ToB managed

Solution A — Azure AI Foundry Host Agent

ToB Managed · fully managed agent runtime · uses BOTH Path 1 and Path 2

GA



Architecture — see legend on the diagram



Solution B — AKS + agent-sandbox

ToB High-Security · Kata Container micro-VM isolation · Path 1 only

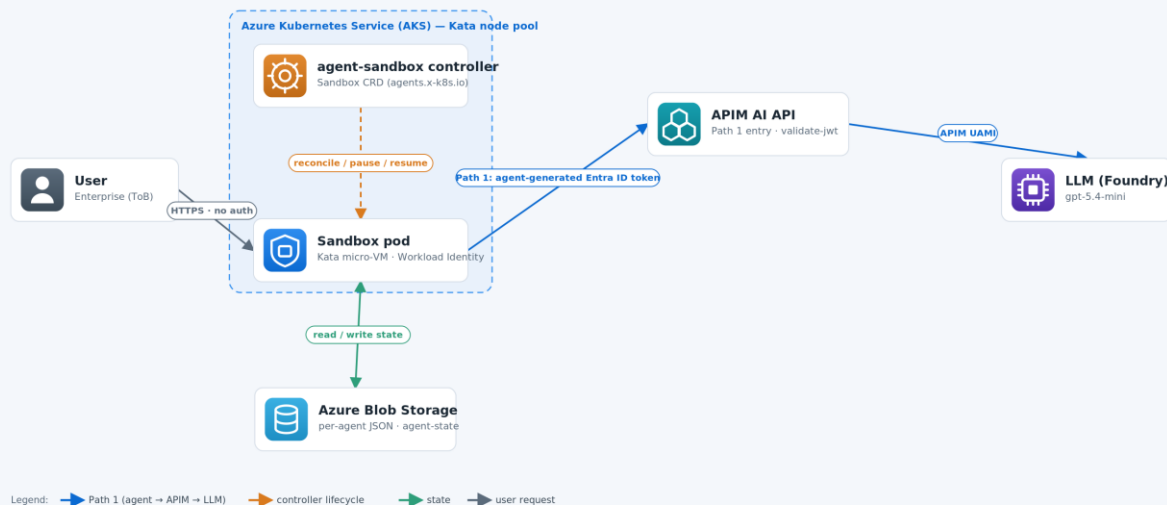
AT A GLANCE

- Strongest isolation — Kata Container micro-VMs
- AKS Pod Sandboxing (kata-vm-isolation)
- agent-sandbox controller: Sandbox CRD
- Lifecycle: pause / resume / hibernate
- Status: GA · Best for ToB high-security

Solution B — AKS + agent-sandbox

ToB High-Security · Kata Container micro-VM isolation · Path 1 only

GA



Architecture — see legend on the diagram



Solution C — ACA Sandbox

ToC / ToB Long-Running · micro-VM boundary isolation · Path 1 only (Public Preview)

AT A GLANCE

- Micro-VM boundary isolation
- Suspend / resume + snapshot state
- Reuses the Module 3 container image
- Per-agent container · UAMI / Workload Identity
- Status: Public Preview · ToC / long-running

Solution C — ACA Sandbox

ToC / ToB Long-Running · OS-level gVisor isolation, no dedicated VM · Path 1 only

Public Preview



Legend: → Path 1 (agent → APIM → LLM) → state → user request

Architecture — see legend on the diagram



Choosing the Right Solution

No universal winner — match the host to your operational profile

A

Fastest managed on-ramp for enterprise?

Foundry Host Agent

B

Maximum control, micro-VM isolation, private networking?

AKS + agent-sandbox

C

Long-running, strong isolation, serverless?

ACA Sandbox

→

One-time / untrusted code execution (code interpreter)?

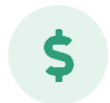
ACA Dynamic Sessions

Full side-by-side comparison in Module 5 — isolation, scale-to-zero, state, Entra ID, APIM, ops complexity & cost.



Cost & Production Hardening

A look ahead to Module 5 — so the labs land in a production context



Cost levers

- Scale-to-zero after 30-min idle
- Blob Cool tier — ~50% cheaper
- Blob lifecycle rules expire stale state
- AKS Spot node pool — up to 90% off (B)
- Azure OpenAI PTU for high volume
- agent-sandbox WarmPool tuning (B)

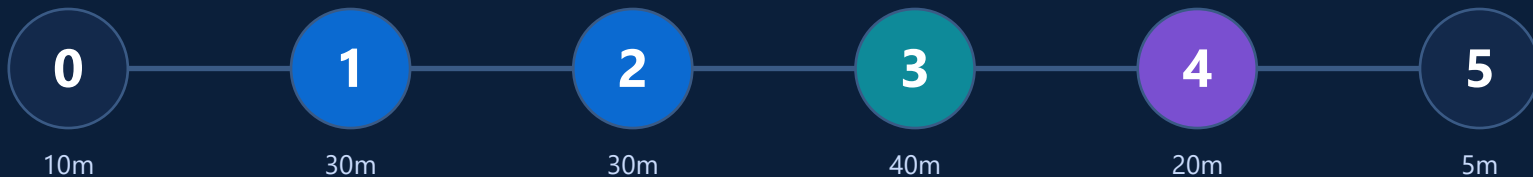


Production hardening

- Private networking (VNet injection)
- Blob versioning + soft delete
- Azure Policy for compliance
- Monitor APIM 4xx / 5xx alerts
- Defender for Containers on AKS
- BCDR: verify state restores from Blob

Let's build.

First stop → Module 1: deploy the shared foundation.



- Every module: one-command deploy or guided manual steps
- All scripts & READMEs are yours to keep
- Questions welcome throughout — total $\approx$ 135 minutes

```
$ cd agenthost/module-01 && ./setup.sh
```